Internal Deep Dive: No Man's Land

File-by-file walkthrough of the DE1-SoC project

Rohit Biswas Kambinachi Obioha Nicola Paparella

CSEE 4840, Spring 2026 — May 13, 2026

> Audience: the three of us, the TA, the professor in a tight Q&A. Goal: you can answer any question about any crevice of this project after reading this once. Pair with `docs/academic_report.md` for the polished framing.

This doc is organised file by file. Each section is the "what is this, what's weird about it, what would break if you changed it" cheat sheet.

Contents

1 Repo layout

```
CSEE4840W-Final-Project/
  README.md            one line; the title
  DESIGN.md            the original design doc with the register map
  SETUP.md             Phase 0/1/2 bring-up
  docs/                this report and the academic one live here
  reports/             LaTeX sources + PDFs (build artifacts in build/)
  hw/                  Phase 1 fabric-only RTL + Quartus project
    de1soc_top.sv      Phase 1 wrapper, no HPS
    nml_gpu.sv         the GPU peripheral (top of nml_gpu's hierarchy)
    avalon_slave_iface.sv
    vga_timing.sv
    sprite_eval.sv
    sprite_fetch.sv
    compositor.sv
    pll_25mhz.v        divide-by-2 placeholder for altera_pll
    gen_rom.py         emits the four .hex files below
    sprite_rom.hex, tile_rom.hex, palette.hex, sprite_table.hex
    quartus/           project + output_files (sof, rbf, reports)
  nml_gpu_hw/          Phase 2 Qsys system + HPS-integrated wrapper
    soc_system.qsys, .qsf, .tcl, .sdc
    soc_system_top.sv
    nml_gpu_hw.tcl
    Makefile           qsys / quartus / rbf / dtb targets
    nml_gpu.sv ...     duplicated from hw/ (KEEP IN SYNC)
  sw/                  C game
    main.c, game.c/.h
    render.c/.h        FPGA renderer
    render_terminal.c  off-board renderer
    nml_gpu.c/.h       /dev/mem mmap driver
    input.h, input_real.c, input_fake.c
    wave.c/.h, autoatk.c/.h
    Makefile
  lab3-reference/      vga_ball lab, kept around for reference
```

`hw/` and `nml_gpu_hw/` carry the same SystemVerilog. **There is no build rule that keeps them in sync.** When you edit `hw/avalon_slave_iface.sv`, you also have to copy it into `nml_gpu_hw/avalon_slave_iface.sv`, or you'll spend an evening hunting a stale-source bug. This is the single biggest piece of foot-gun in the project; fix it before next semester (symlink or `make sync`).

2 Phase 1 vs Phase 2

- **Phase 1.** Top-level entity is `de1soc_top`. Avalon slave is tied off. The bitstream alone is a complete VGA producer thanks to `$readmemh` initialisation of the palette, sprite table, sprite ROM, and tile ROM. Compiled from `hw/quartus/nml_gpu.qpf`. Output is `nml_gpu.sof` (JTAG) and `nml_gpu.rbf` (SD card). Use this to verify the fabric pipeline in isolation.
- **Phase 2.** Top-level entity is `soc_system_top`. The HPS is on the board, a Platform Designer system bridges HPS-LW to nml_gpu's Avalon slave, the device tree exposes the peripheral, and the C binary `mmap()`s `/dev/mem` at 0xFF200000. Compiled from `nml_gpu_hw/`. Output is `soc_system.rbf` + `soc_system.dtb` for the SD card.

The hot-loop for Phase 1 bring-up is `quartus_pgm -m JTAG ...` (or double-click in the GUI). The hot-loop for Phase 2 is mount SD card → `cp soc_system.rbf soc_system.dtb /Volumes/<boot>/` → eject → boot → `scp` the binary → run. We do most Phase 2 iteration native on the board (`make native` on the device) because that beats the SD-card shuffle.

3 `hw/nml_gpu.sv` — top of the GPU hierarchy

305 lines. Two clock domains: `clk` (50 MHz, Avalon) and `pix_clk` (25 MHz, VGA). The full memory inventory and most of the inter-block glue lives here, not in the slave.

- Lines 22, 38: tie `vga_sync_n = 0` (DAC uses the dedicated sync pins, not sync-on-green); invert `vga_clk = ~pix_clk` so the ADV7123 samples mid-cycle. **If you stop inverting, expect colour fringing or wrong sampling — you are letting the DAC sample during the FPGA's transition window.**
- Lines 29–33: PLL instance. `outclk_0` is `pix_clk`. Reset is inverted (`pll_25mhz` is active-high; the project's reset is active-low).
- Lines 73–87: sprite table active/shadow. Two 32×64 RAMs. Avalon writes go to shadow (split into low/high 32-bit halves by `mem_waddr[0]`), Avalon reads come from shadow.
- Lines 94–118: cross `ctrl_swap_req` from `clk` into `pix_clk`. The third synchroniser flop is for edge detection; you set `swap_pending` on the rising edge of the synchronised request, bulk-assign the active table from shadow on the next vsync, and clear `swap_pending`. Three points of subtlety:
  1. `swap_pending` is in the pix_clk domain. The slave reads it back via `swap_pending_in` on the Avalon clock — this is a CDC the other way. The only reader is a polling loop slower than 50 MHz so the 1–2 cycle race is harmless.
  2. The bulk assignment loop synthesises as 32 parallel registers, not a sequential loop. 2048 flops on one edge. If you grew the table to 256 entries this would blow up.
  3. If you remove the edge detect, you reload `swap_pending` every pix_clk cycle the synchroniser sees the request high — redundant but harmless.
- Line 124: `eval_sprtab_rdata = sprite_table_active[eval_sprtab_raddr]` is **combinational** on pix_clk. sprite_eval drives `raddr` and expects `rdata` next cycle through.
- Lines 132–136: palette RAM. Avalon-side write port at `clk`, pix_clk-side read port.
- Lines 138–173: tile map. Read this section twice if you're going to edit it. The packed `[3:0][7:0]` form is the only thing that makes byte writes work over the LW bridge.
- Lines 176–195: ROMs and initialisers. `$readmemh` paths are **relative**. They resolve against the directory Quartus is run from (i.e. `hw/quartus/`). The QSF adds `SEARCH_PATH ..` to rescue them. If you move the QSF or run `quartus_sh` from elsewhere, the ROMs come up zeroed and the FPGA emits a black screen.
- Lines 198–221: dual line buffers. `linebuf_sel` toggles at `hblank && x == 640`. The "fill" buffer is whichever index matches `linebuf_sel`; the compositor reads the *other* one (line 221). **If you flip the polarity of `linebuf_sel`, the compositor reads garbage.**

- Line 248: `eval_strobe = (x == 642)`. The 642 is arbitrary — any value in $[640, 700]$ would work.
- Lines 250–270: the `eval_done_r` sustain latch. The reason this exists is in the comment block at lines 259–263 and in the academic report Section 9.2. **Do not remove unless you replace it with a more reliable handshake.**
- Lines 298–300: `vga_r/g/b` mux on `ctrl_enable`. When disabled, all three are 0 and the monitor goes black. Used at shutdown (`nml_set_enable(0)` in `main.c:270`).

Listing 1: SWAP cross-clock-domain machinery (`hw/nml_gpu.sv:94-118`).

```
always_ff @(posedge pix_clk or negedge reset_n) begin
    if (!reset_n) begin
        ctrl_swap_req_sync_a <= 1'b0;
        ctrl_swap_req_sync_b <= 1'b0;
        ctrl_swap_req_sync_c <= 1'b0;
    end else begin
        ctrl_swap_req_sync_a <= ctrl_swap_req;
        ctrl_swap_req_sync_b <= ctrl_swap_req_sync_a;
        ctrl_swap_req_sync_c <= ctrl_swap_req_sync_b;
    end
end
wire ctrl_swap_req_pulse = ctrl_swap_req_sync_b && !ctrl_swap_req_sync_c;

always_ff @(posedge pix_clk or negedge reset_n) begin
    if (!reset_n) swap_pending <= 1'b0;
    else begin
        if (ctrl_swap_req_pulse) swap_pending <= 1'b1;
        if (swap_pending && vsync) begin
            for (int i=0; i<32; i++) sprite_table_active[i] <= sprite_table_shadow[i];
            swap_pending <= 1'b0;
        end
    end
end
```

4 `hw/avalon_slave_iface.sv` — register decode

268 lines. The header comment block at lines 1–22 is the canonical register map.

- Lines 95–96, 199–209: SWAP pulse stretcher. 3-bit counter, loaded with 7 on each write to CTRL bit 1. Holds `ctrl_swap_req` high for seven 50 MHz cycles (140 ns). **If you shrink the count below 3 you risk losing SWAPs.**
- Lines 115–123: address decode. `region_palette` looks weird (the second clause is logically redundant); intent is "0x400–0x7FF inclusive".
- Lines 133–141: `avs_waitrequest` is held high on the first cycle of any RAM-region read, then released. **If you tie waitrequest to 0 unconditionally, the master will sample the previous `*_rdata_sw` instead of the one for the current address.**

- Lines 156–179: write-decode `always_comb`. `mem_waddr` is computed for both reads and writes (the `if (region_*) ...` does not gate on `avs_write`). This matters because the read path shares the same `mem_waddr` to address the SW-readback registers.
- Lines 169–178: **tile-map address packing**. The expression `{avs_address[13], avs_address[11:0]}` is non-obvious. The tile map spans 0x1000–0x22BF; for 0x1000–0x1FFF, bit 13=0 and bit 12=1 so the packing yields 0x000–0xFFF; for 0x2000–0x22BF, bit 13=1 and bit 12=0 so the packing yields 0x1000–0x12BF. Truncating to `[12:0]` (the old behaviour) aliased the upper half on top of the lower half and clobbered rows 0..8 every time row 24+ was written. **Touching this expression risks the stripe bug coming back in a different form.**
- Lines 184–230: scalar-register write `always_ff`. CTRL has the auto-clear on bit 1 (line 208). **`frame_ctr_in` is hardwired to 8'd0 at the top level (line 231) — the frame counter in STATUS[15:8] is always zero.** Wire it up to a vsync-edge counter as a Phase 3 task.
- Lines 235–266: read mux. Combinational. Default `32'h0` if no region matches.

5 `hw/vga_timing.sv` — 800×525 frame walker

64 lines. The standard 640×480 @ 60 Hz parameters.

- `h_count`, `v_count` reach 799 and 524 at the wrap line. `x, y` are direct aliases.
- `hsync`/`vsync` are **active low**, generated as `~(in_sync_range)`.
- `hblank = (h_count >= 640)`. `vblank = (v_count >= 480)`. `visible = !hblank && !vblank`.
- Off-by-one trap: when `x = 799, y = 524`, `tile_col = 99, tile_row = 65`, address is $65 \times 80 + 99 = 5299$. The tile map only has 4800 entries. The spurious accesses are during blanking, where `VGA_BLANK_N = 0` so the result is masked. **If you ever drop the blanking gate, this reads out of bounds and the porches will show garbage.**

6 `hw/sprite_eval.sv` — top-8 priority sort

117 lines. FSM with four states.

- Lines 32–34: bit positions in the 64-bit sprite word for `y` (31:16), `prio` (46:44), `active` (47). Bit 47 is where the C driver's `NML_FLAG_ACTIVE` ends up after the `nml_write_sprite` packing (`sw/nml_gpu.h:69`, `sw/nml_gpu.c:106-109`). **The design doc said sprites are hidden by sentinel coordinates; the hardware reads bit 47 as the active flag. The C driver hides via both paths (sets $x = y = -256$ and clears bit 47).**
- Lines 77–94: insertion sort. The `inserted` variable is a `logic` declared inside the always block; it is combinational inside the always edge. Each slot's compare runs in parallel; the `if (!inserted)` chain forces sequential semantics so a single sprite lands in only one slot. Total: ~512 LE gates of compare and shift.
- Line 97–102: `sprite_idx` is 6 bits so it can naturally reach 32. When `sprite_idx == 31`, state goes to `DONE`; on the next cycle, the sorted list is latched and `eval_done` is pulsed.

- Total cycle budget: 32 fetches + 32 evals (interleaved) + 1 DONE = ~65 cycles. HBLANK at 25 MHz is 160 cycles; eval finishes with margin.
- **Quirk:** `top_sprites/top_prios/top_valid` are not declared with explicit reset values; they survive between scanlines unless the IDLE-to-FETCH transition clears them. Lines 60–62 do that clear. If you move the clear elsewhere, you get a slow death where stale sprites linger across scanlines.

7 `hw/sprite_fetch.sv` — ROM read + line buffer write

135 lines. FSM with six states.

- Lines 32–38: bit positions for `x` (15:0), `y` (31:16), `id` (37:32), `hflip` (42). `vflip` is at bit 43 but not used — only hflip is wired in the inner address compute.
- Line 42: `pixel_v = next_scanline - spr_y`. Signed subtract; the result is the row within the 16-pixel sprite.
- Line 75: in CLEAR_BUF, after writing pixel 639 we **wait** until `eval_done` before transitioning.
- Lines 87–97: walk sprites high to low priority. The pixels of the highest-priority sprite are written *last*, overwriting any lower-priority sprite at the same X. Inactive sprites (active_mask bit clear) are skipped in one cycle each.
- Lines 99–114: READ_PIXEL drives `sprrom_raddr`, then we wait one cycle in WAIT_PIXEL for the ROM read to complete, then WRITE_PIXEL writes the result if it's non-zero and on-screen. The flow takes 3 cycles per pixel, so 48 cycles per sprite if all 16 pixels are non-transparent.
- **Cycle budget arithmetic for the worst case:**
  - CLEAR_BUF: 640 cycles.
  - 8 sprites × 48 cycles = 384 cycles for fetch.
  - Total: ≈ 1024 cycles needed.
  - HBLANK: 160 cycles at 25 MHz.

  We cannot finish the fetch within the HBLANK that precedes its scanline. **The reason the pipeline works at all is that the line buffers are double-buffered with a 1-line latency:** while we fetch into the active fill buffer during scanline N's HBLANK and N's visible time, the compositor reads from the other buffer for scanline N. The fill is allowed to spill over into the visible region; what matters is that it finishes before the swap at the end of scanline N. The real budget is HBLANK (160) + visible (640) = 800 cycles, which is just enough for the worst case. **A 9th sprite per scanline would not fit.**

8 `hw/compositor.sv` — pipeline + HUD

313 lines. Four pipeline stages.

Things worth memorising for the Q&A:

- HUD strip is the top 16 pixels ($y < 16$). HP bar at $x = 0..191$ (192 px wide). Then AMMO/WAVE/ART/GAS/SCORE labels and digits. Lines 36–66 are the layout constants; lines 70–73 are the HUD colours.
- Lines 99–101: HP bar fill is `hp_x2 = hp` $\ll$ `1`, clamped to 192, with per-pixel fill flag `(x[8:0] < hp_bar_width)`. HP is 0..100, so `hp_x2` is 0..200; clamp catches the upper bit.
- Lines 127–178: label tile lookups. Each label region maps a character index (derived from local-$x \gg 3$) to a tile ID in the letter band 16..41 (`16 + (letter - 'A')`).
- Lines 180–212: BCD digit extraction. Score is 6 BCD digits in `score_reg[23:0]`. Leftmost screen column is the highest digit, so `score_nibble_idx = 5 - score_digit_idx`.
- Lines 240–244: background tile map address. `tile_col = x[9:3]`, `tile_row = y[9:3]`. **Implicitly assumes the visible region is the entire 80×60 grid.** `scroll_x`/`scroll_y` are routed in but currently unused — adding scrolling means adding the scroll values into `tile_col`/`tile_row` here and re-validating wrap behaviour.
- Lines 247–268: S1→S2 latches. Important to register the background tile map data here so the tile ROM address (in S2) has one cycle to settle. **`sprite_pixel_delay <= linebuf_rdata`** is the latched copy of the sprite line buffer value.
- Lines 271–273: tile ROM address mux. If we are drawing HUD text, the tile id comes from the HUD path; otherwise from the background.
- Lines 280–286: S2→S3 latches. Just delaying the HUD-region flags one more cycle.
- Lines 289–292: palette index select. Sprite pixel wins if non-zero; otherwise tile pixel.
- Lines 296–311: final RGB. If not visible → black. If HP bar → green or dark red. If HUD text → white on dark grey (tile ROM glyph byte is 0xFF for lit pixels, 0x00 for blank). Else → `palette_rdata`.

Pitfall: the `s3_in_hud_*` flags lag the actual tile ROM read data by one cycle. If you ever change the pipeline depth, recheck that every HUD-region flag is delayed by exactly the right number of stages.

9 `hw/pll_25mhz.v` — divide-by-2

30 lines. Register `clk_div` toggles on every 50 MHz rising edge. Quartus auto-promotes the register output onto a global clock network. Works on silicon, fails analyser pulse-width check.

Replace with `altera_pll` in Phase 3. The port names are the defaults so it's a drop-in. Until then, accept the negative slack in the timing report.

10 `hw/gen_rom.py` — ROM emitter

777 lines. Three functions worth knowing about.

Sprite IDs:

ID	Sprite	Notes
0	Transparent	All bytes zero
1	Player	Green-bordered square
2	Armed enemy	Red with white cross
3	Player bullet	4×4 yellow centred
4	Unarmed enemy	Solid pink
5	Mortar shell	Orange diamond
6	(retired) wire	Removed when AA_WIRE dropped
7	Mustard gas	Green circle radius 7
8	Artillery flash	White plus, 7-pixel arms
9	Ammo crate	Green crate outline
10	Enemy bullet	Red dot radius 3

Tile slots in use: 0 (blank background), 4–5 (dirt), 6–7 (grass), 8 (dirt/grass transition), 9 (mud puddle), 10 (dense grass), 11 (artillery beam column), 16–41 (A–Z glyphs), 42 (colon), 43 (space), 48–57 (digits 0–9). Glyphs use palette index 0xFF for lit pixels (white).

Palette: 256 RGB888 entries; **only some are populated; the rest are zero (black).** The active slots are documented inline in `sw/main.c:139-168`, which mirrors `make_palette()`. **If you change a palette index here, also change `sw/main.c`** — the C runtime overwrites the `$readmemh` init the moment `nml_open()` finishes.

The script is invoked manually (`python3 hw/gen_rom.py`) before synthesis. **It is not driven by the QSF** — if you change a sprite or palette, you have to regenerate the hex files and re-synthesise.

11 `sw/nml_gpu.c` / `sw/nml_gpu.h` — userspace driver

191 lines (`.c`) + 108 lines (`.h`).

- `nml_open()` (`sw/nml_gpu.c:33-59`) opens `/dev/mem` with `O_RDWR | O_SYNC` and `mmap`s 16 KB at `NML_LWFPGA_BASE = 0xFF200000`. **Both `O_SYNC` and the `mmap` flag matter:** they ensure that writes are not cached by the ARM L1.
- `reg_write()`/`reg_read()` use `volatile uint32_t *`. **Removing `volatile` will let gcc optimise away repeated polls of `STATUS`.**
- `nml_write_tile()` (lines 92–99) does a `volatile uint8_t *` write. This is the `STRB` that hits the byte-enable path on the slave. **If you change this to a 32-bit write with a mask, you trigger the bridge's RMW behaviour and the same lane problem returns.** Keep it as a byte write.
- `nml_write_sprite()` (lines 101–113) packs the 8-byte entry as two 32-bit words, low half then high half. `flags` lives in W1[15:8]; `NML_FLAG_ACTIVE` ends up at bit 47 of the 64-bit packed word.
- `nml_hide_sprite()` (lines 115–125) is belt-and-braces: writes the sentinel coordinates *and* clears the ACTIVE bit.

- `nml_commit_frame()` (lines 179–190) sets `SWAP` and polls `SWAP_PENDING` for up to 200,000 reads. **If the FPGA never clears SWAP_PENDING, we log a timeout and continue, rather than hang.**
- `bcd_pack()` (lines 133–140) packs a small integer into 4-bit nibbles, low nibble = ones digit. Used for the score, wave, ammo, and charge HUD fields.

12 `sw/main.c` — main loop

275 lines.

- Lines 28–33: SIGINT/SIGTERM handler sets `g_running = 0` so the loop exits cleanly. On exit, video is disabled (`nml_set_enable(0)`) and `/dev/mem` is unmapped. **Don't kill the binary with `-9`; you leave the video on with stale sprites.**
- Lines 139–168: palette init. Sixteen entries. The values must match `hw/gen_rom.py` `make_palette()` or the FPGA's `$readmemh`-initialised palette will be overwritten with bogus colours.
- Lines 195–265: main loop body. Read input → tick → render → commit → pace. Frame target is 16,666 μs (60 Hz).

13 `sw/game.c` / `sw/game.h` — state machine + entity pool

Worth memorising the input bits because they appear in `main.c`, `game.c`, and both input readers.

- `entity_t` (`game.h:79-90`) has nine fields. `phase` is a per-spawn random byte used only by enemy AI. `ttl` and `ttl_max` drive auto-projectile/hazard lifetimes (and the gas grow/shrink animation phase). `payload` carries which auto-attack kind spawned an `ENT_AUTO_PROJ` or `ENT_HAZARD`.
- `game_init()` (`game.c:64-90`) spawns the player at `(SCREEN_W/2, SCREEN_H - 60)` with HP 100. Initial ammo 40, art 2, gas 2. Calls `wave_system_init()` and `autoatk_init()`.
- `update_player()` (`game.c:138-165`) clamps the player into $[0, \text{SCREEN\_W} - 16] \times [\text{HUD\_H}, \text{SCREEN\_H} - 16]$. `HUD_H = 24` — **the player can never enter the top 24 pixels.**
- `update_enemies()` (`game.c:219-254`) calls `enemy_rng()` every frame. The window is 12 frames (`ENEMY_DIR_FRAMES`). **If you change `ENEMY_DIR_FRAMES`, expect the enemy feel to shift dramatically.**
- Lines 248–252: enemy steps over the trench (`y >= SCREEN_H - 20`), deactivates, costs the player 10 HP. **No score for these kills.**
- `handle_collisions()` (`game.c:339-412`) is the only place that decides HP loss, score gain, kill counter, ammo drop. **Routes through `apply_damage()`** for enemy kills.
- Lines 416–426: LEVELUP cursor handling. D-pad left/right wraps modulo `LEVELUP_OPTIONS`. B (the down-fire button) is the confirm. **Hijacked from the four-direction fire system — `INPUT_FIRE` is now aliased to `INPUT_FIRE_DOWN` (`sw/game.h:31`).**

14 `sw/wave.c` — wave table

120 lines. Just the table and the spawner. Five waves, clamps after wave 5.

To rebalance the game, edit the `WAVES[]` table and rebuild. No runtime file I/O, no JSON, no waves.dat. The original DESIGN.md described a waves.dat file; we shipped a hard-coded table for simplicity. Change the table size and `WAVE_COUNT` updates automatically.

15 `sw/autoatk.c` — auto-attacks

274 lines.

- Mortar (lines 73–107) is the only timer-driven weapon. `BASE_PERIOD[AA_MORTAR] = 180`, step −30 per level, floor 60.
- Artillery (lines 200–219) is player-triggered. Kills every enemy in a 16-pixel column above the player by passing `ARTILLERY_KILL_DMG = 999` through `apply_damage()`.
- Gas (lines 224–242) is player-triggered. Spawns ONE `ENT_HAZARD` at the player position. The renderer fans this out to up to four sprite slots based on the cloud's current size, but on the game-logic side there is exactly one hazard entity with one hitbox.
- `autoatk_pick_levelup_options()` (lines 244–264): rejection sampling for 3 distinct weapon kinds out of `AA_COUNT = 3`. Always offers all three (with order shuffled).

Common Q&A: "How does the player upgrade a weapon they don't own?" — every weapon starts at level 0 in `autoatk_init()`, and `autoatk_upgrade()` bumps them up. Level 0 is "not owned and inert" (mortar's `cooldown` only decrements if `level > 0`, line 103). Picking a level-0 weapon at LEVELUP gives it level 1 and a fresh cooldown.

16 `sw/render.c` — entity to sprite-table writer

422 lines.

- Lines 21–44: sprite_id and tile_id constants. **These must match `hw/gen_rom.py` exactly.** Any drift here is the second-worst foot-gun in the project.
- `render_init_tilemap()` (lines 146–152) paints the entire battlefield using `tile_for(row, col)` (a deterministic noise function). Called once at startup and again after every game-over → restart. **Slow:** 4800 tile writes, each a byte STRB across the LW bridge. Takes a few ms on the board.
- Lines 161–189: artillery beam paint + restore. **No per-cell shadow.** This works because `tile_for(row, col)` is deterministic — a hidden invariant. If you ever make the tilemap stateful (e.g. craters from mortar hits), the beam restore will erase them.

- Lines 230–283: `render_levelup`. Slot 0 = player, slots 1..3 = three weapon-sprite options at fixed positions (x = 160, 320, 480; y = 100), slot 4 = the cursor sprite above the chosen option. Slots 5..31 are hidden.
- Lines 302–340: `render_game_over`. Overwrites the tile map with five lines of centred text. **This is the screen we know is broken in the current Phase 2 build.**
- Lines 342–421: `render_frame()`. On GAMEOVER → PLAYING, calls `render_init_tilemap()` to restore the ground.

17 `sw/render_terminal.c` — host renderer

173 lines. Replays the slot-allocation logic from `render.c` but prints each sprite slot as an ASCII glyph (P for player, A for armed enemy, u for unarmed, * for bullet, ^ for enemy bullet, + for ammo drop, M for mortar, | for artillery, ~ for gas, etc.).

Used by `make terminal`, which also defines `NML_TERMINAL_BUILD` so all the FPGA-only code in `main.c` and `render.h` is compiled out. Frame pacing is `nanosleep` only.

18 `sw/input_real.c` — evdev reader

176 lines.

- Lines 45–58: button code map. **Verified on this specific KIWITATA SNES-style DragonRise pad (VID 0079:0011). If a different pad is used, set `NML_INPUT_DEBUG=1` and watch stderr for the raw event codes.**
- Lines 76–84: axis range query via `EVIOCGABS`. Cache min, max, centre, and initial value at open time. Some pads report 0..255, some −32768..32767, some −1..0..1. The deadzone threshold is range/4.
- Lines 127–147: drain events nonblocking. Updates the persistent `button_state` bitmask and the axis values. **No threading; events are drained on every `input_read()` call.**
- Lines 87–97: if `open("/dev/input/event0")` fails, we set `ev_fd = FD_DISABLED` and return zero input forever. The game keeps running without controller input — useful for bring-up debug.

19 `sw/input_fake.c` — stub

44 lines. A fixed cycle over 480 frames that exercises every direction and every face button. Used in `make terminal` and `make fake-input`. **Tip:** if you want a different fake cycle, edit this file rather than smudging it into `input_real.c` with `#ifdef`s. The two implementations are entirely separate.

20 sw/Makefile

73 lines. Targets: default (cross-compile for armhf with `input_real.c`), `native` (compile on the board), `fake-input` (cross-compile for armhf with `input_fake.c`), `terminal` (host build with `render_terminal.c` and `input_fake.c`), `clean`.

Cross-compiler: `arm-linux-gnueabihf-gcc`. `-static` everywhere so we don't have to match glibc versions against the board's rootfs.

21 Boot and run on the board

The full procedure is in `SETUP.md`. Cliffs Notes:

1. Build the bitstream from `nml_gpu_hw/` (`make qsys && make quartus && make rbf`).
2. Build the DTB (`make dtb`).
3. Mount the SD card, copy `soc_system.rbf` and `soc_system.dtb` onto the FAT boot partition (exact names required), eject.
4. Boot. Linux comes up at `login:`. Username `root`, no password.
5. `ls /proc/device-tree/sopc@0/bridge@0xc0000000/` should show `vga@0x100000000`.
6. `scp sw/nml_game root@<board>:/root/` and run it.

22 Known bugs and fragile spots (as of 2026-05-12)

- **Stripe rendering bug.** The Phase 2 build shows vertical stripes across the entire screen. Investigation pointed at the tile-map byte-enable path (see academic report Section 9.5). The packed-byte storage in `hw/nml_gpu.sv:150-173` and the byte-address packing in `hw/avalon_slave_iface.sv:169-178` are the two surfaces in scope.
  Untested hypothesis: the read-side byte select (`tilemap_word_pix[tilemap_raddr_lo_r]`) is correct, but the *write-side* address mapping in `avalon_slave_iface` sets `mem_waddr = {avs_address[13], avs_address[11:0]}` which is a 13-bit value indexing a 1200-deep array — the top bit at `mem_waddr[12]` is being passed through, but the `tilemap_ram` index in `hw/nml_gpu.sv:161` uses `mem_waddr[12:2]` to get a word index. The result is a 11-bit word index, range 0..2047. The tile map has 1200 words. **Writes to word indices > 1199 may be silently ignored (or worse, alias).** Action item: verify the bridge cannot drive `avs_address` outside the declared region, or clamp it explicitly.
- **Broken Game-Over screen.** Likely the same root cause as the stripes — the tile-map writes for the centred text don't all land where they should.
- **`$readmemh` paths are relative.** Synthesise from a different cwd and the ROMs come up zeroed.
- **`STATUS[15:8]` frame counter is hard-coded to 0** in `hw/nml_gpu.sv:231`. Not a bug per se; just unconnected.
- **Sprite fetch can't finish all 8 sprites within a single HBLANK.** Works only because the line buffer is double-buffered — the fetch can spill into the visible region of the previous line. A 9th sprite per scanline would not fit.

- **Compositor tile address goes out of range during porches.** `tile_row` reaches 65 at $y = 524$; tile map has 60 rows. Currently masked by `VGA_BLANK_N = 0`. Don't remove the blanking gate.
- **No active bit was in the design doc;** the hardware reads bit 47 as an explicit ACTIVE flag. The C driver hides via both paths.
- **`sw/main.c` palette init must match `hw/gen_rom.py`.** No build-time check.
- **Quartus timing report shows -5 ns slack on `pix_clk`.** False positive — the divide-by-2 register is promoted to a global clock and works on silicon. Ignore until the `altera_pll` swap.

23 What would break if changed

- Remove `vga_clk = ~pix_clk` inversion → DAC samples on the FPGA's transition, colour fringing.
- Remove the `eval_done_r` sustain latch → fetch starves on 15 of 16 scanlines.
- Unpack the tile-map RAM to `[7:0] [0:4799]` → byte writes from the bridge clobber three lanes per word.
- Remove `avs_waitrequest` on RAM reads → master samples stale `*_rdata_sw`.
- Shrink `swap_req_cnt` below 3 → pixel-clock synchroniser misses SWAP requests.
- Move `$readmemh` files or change Quartus cwd without updating `SEARCH_PATH` → ROMs come up zeroed.
- Change the sprite IDs in `render.c` without updating `gen_rom.py` → entities draw with the wrong sprite art.
- Change the palette in `gen_rom.py` without updating `sw/main.c init_palette_runtime()` → palette init at startup overwrites the correct values with the C-side stale copies.
- Drop the `VGA_BLANK_N` gate → out-of-range tile addresses leak garbage onto the porches.
- Drop `MAP_SHARED | O_SYNC` on the `/dev/mem` open → ARM L1 caches MMIO and the SWAP polling loop never sees the FPGA's view.
- Remove `-static` from `sw/Makefile` → binary fails to link on the board (glibc version mismatch).

End notes

If a professor asks you about a specific module, open the file in this doc, walk them through it line by line. The cycle counts in Section 6 (eval) and the worst-case math in Section 7 (fetch) are the two numbers most likely to come up. The "why the slave widens SWAP" story in Section 4 / 9.3 of the academic report is a good example of CDC discipline if asked to defend the design.

For the stripe bug specifically: **own it.** It is a real bug, the hypothesis is articulated, and the next-step diagnosis is in the academic report. The professor will respect a documented unknown more than a confident hand-wave.